MANTHAN 2026

Autonomous Trading Bot

Judges' Briefing

What to explain, how every part works, and the questions the jury is likely to ask (with the answers we should give)

Team	Tit4Tat (portal user @mn2602, Kafka group group-mn2602)
Challenge	JNNCE MANTHAN 2026: Autonomous Trading Systems Challenge
Code	manthan-bot (Python), private GitHub repo vishwasvishubm/manthan-bot
Starting capital	₹10,00,000, 10 masked stocks, 500 simulated trading days
Date of this briefing	30 October 2026

Our one-line pitch: a disciplined, explainable, zero-error bot that puts the money to work on day 1, keeps costs and taxes close to zero, and only changes its plan when the evidence says so.

Contents

1. The 2-minute pitch (say this first)
2. Our results at a glance
3. What the bot does on one simulated day
4. Architecture and modules
5. How each part works
6. How we chose the strategy (the evidence)
7. Testing and verification
8. Live demo plan for the judges
9. Questions the judges may ask, with our answers
10. Honest limitations and what we would do next
11. Glossary (every term on the leaderboard and in this document)
12. One-page numbers cheat sheet

Tip for the presentation: memorise section 1 and section 12. Sections 5 and 9 are for answering questions in depth. Always give the honest answer when a judge finds a weakness: section 10 lists them so nothing catches us by surprise.

1. The 2-minute pitch (say this first)

Speak these points in this order. Each one is backed by a number we can show.

- **What it is.** A fully autonomous Python bot. It reads the live market and news stream from Kafka, decides once per simulated day in about 0.35 seconds, and trades through the Order Gateway. No human touches it after start.
- **What it does with the money.** On day 1 it spreads 99% of the capital equally over all 10 stocks and holds them. A 30% concentration cap stops any single stock from taking over the portfolio, and a tax engine times every sale to pay the lowest legal tax.
- **Why that strategy.** We did not guess. We tested every idea (trend timing, mean reversion, momentum, news, short selling, volatility weighting) on all 32 possible 500-day games in the data. Every timing layer lost money after costs. The disciplined core won: mean +101.5%, worst game +42.6%.
- **Live proof.** In the SimTest live run we finished **2nd on the leaderboard**: wealth ₹14,40,868 (+44.1%), **0 errors**, **0 s downtime**, 0.51 s average decision, only ₹1,630 charges and **₹0 tax** on ₹77,058 of realized profit.
- **Engineering quality.** Every order is pre-checked against our own copy of the account, so it cannot be rejected. Timeouts are handled without ever double-ordering. 84 automated tests and a 22-point verification script pass. Our backtest reproduces the live run within 0.14%.
- **Explainable.** Every decision and every order is logged with its reasons, sizing and risk checks, so we can show the jury exactly why the bot did anything.

Closing line: "Most bots try to be clever every day. Ours is clever about *when not to trade*. That is why it had zero errors, near-zero costs, zero tax, and still finished 2nd."

2. Our results at a glance

2.1 The live SimTest run (official leaderboard of the pre-flight / test simulation)

Metric	Our value	What it shows
Rank	2nd	Top-2 out of all teams
Total wealth	₹14,40,868.47	+44.1% on ₹10 L
Realized profit	₹77,058.03	Profit from stocks actually sold
Unrealized profit	₹3,65,453.36	Gain on stocks still held (not taxed until sold)
Charges	₹1,629.52	Lowest among the active teams; others paid ₹10k–20k
Taxes	₹0.00	All realized gain was exempt or offset (see section 9C)
Cash utilization	88.5%	How much of the money was invested on average
Orders / errors	60 / 0	Every single order accepted
Downtime	0 s	Never disconnected
Average decision time	0.51 s	Well inside the 10 s day
POS (positions at end)	10	Fully diversified: one position in every stock

For comparison: the 1st-place team held only 3 stocks, realized ₹7.34 L of profit mostly short-term, and paid ₹63,435 in tax. Other teams placed 400–723 orders; one had 423 errors.

2.2 Since the live run: the improved bot, replayed on the same 500 days

Run	Wealth	Realized	Unrealized	Charges	Tax	Util	Orders	Errors	POS
Live run (portal)	14,40,868	77,058	3,65,453	1,630	0	88.5%	60	0	10
Replay, settings that ran live	13,33,500	36,231	2,98,385	1,116	0	90.3%	42	0	10
Replay, current bot	14,36,954	48,157	3,89,032	235	0	94.8%	11	0	10

- The live run was a little lucky: a data glitch (now fixed) delayed 5 stocks by about 5 days and that delay happened to pay off. When we emulate that glitch, the replay gives ₹14,38,801, within 0.14% of live, which

proves our simulator matches the platform.

- The fair comparison is replay against replay: the current bot is about **+10 points** better than the settings that ran live (+43.7% vs +33.4% after tax), with 11 orders instead of 42 and ₹235 of charges.

2.3 Every possible 500-day game in the data (32 episodes)

Version	Mean return	Median	Worst	Mean max drawdown	Mean util	Mean orders	Mean tax
Current bot (99% core, 30% cap)	**+101.5%**	+72.6%	**+42.6%**	24.9%	98.7%	23	₹4,658
97% core, 2% cash, 6 orders/day	+99.9%	+71.7%	+41.6%	24.6%	97.2%	22	₹4,287
As run live (80% core + tactical)	+93.0%	+70.9%	+30.1%	24.0%	93.7%	47	₹4,704

Each episode is 500 trading days (like the real game), starting every 10 days from 2023-07-27 to 2024-10-18, with 400 days of history before it (what the bot gets live), recorded news replayed, and day 1 only observed (as happens live). Returns are after tax.

3. What the bot does on one simulated day

The platform publishes one daily candle per stock on Kafka every 10 seconds (one simulated day), plus news headlines. This is the bot's routine for each day:

Step	What happens	Where in code
1. Collect	Ticks for all 10 stocks arrive on market.ticks; headlines on market.news. The engine batches them by date.	stream.py, engine.py
2. Wait smartly	It decides as soon as every stock's tick is in and no news has arrived for 150 ms (so late headlines are not missed).	engine.py
3. Indicators	Updates EMAs, ATR, RSI, Bollinger z-score, efficiency ratio, momentum and volatility for each stock from its full history.	indicators.py
4. Score and rank	Gives every stock an explainable score in [−1, +1] and ranks them by relative strength. (In the default profile the score is logged for explanation; trading is driven by the core.)	strategy.py
5. Risk check	Measures drawdown against the 60-day peak and the one-day loss; reads the STOP / FLATTEN kill-switch files.	risk.py
6. Plan orders	Builds at most 10 orders: core buys to target, concentration-cap trims, March tax-loss harvest, exits before entries. Each order is pre-validated against the local ledger.	planner.py
7. Execute	Sends orders in parallel (up to 10 workers) through the REST gateway; books fills and partial fills; investigates any unknown outcome.	engine.py, api.py
8. Journal	Writes the day's decision (every score, reason, risk state, latency, wealth) and every order's explanation to logs/*.jsonl; updates the dashboard snapshot.	journal.py

Separately and all the time: a stage watcher polls GET /stage every 2.5 s (trading only when the stage is **simulation / RUNNING**), and a heartbeat worker answers every pre-flight heartbeat in order.

4. Architecture and modules

Layer	Component	Job
Input	Kafka consumer (kafka-python)	Reads market.ticks and market.news (10 partitions) as group-mn2602; auto-reconnect with back-off; starts from the latest offset; no auto-commit.
Input	StageWatcher (lifecycle.py)	Polls /stage every 2.5 s; turns trading on only in simulation/RUNNING; tolerates slow polls for 10 s.
Input	HeartbeatWorker (lifecycle.py)	ACKs every pre-flight heartbeat in sequence (POST /bot/heartbeat), drops repeats, retries failures, reads /bot/status for readiness.
Brain	Engine (engine.py)	One thread and one portfolio state: batching, decisions, execution, duplicate guard, circuit breaker, new-stock discovery, pre-flight to simulation switch.
Brain	Strategy + planner	Scores, ranking, core sizing, concentration cap, tax logic, stops (in tactical profiles), and a written explanation for each order.

Layer	Component	Job
Safety	Risk guard + universe screen	Drawdown and daily-loss limits, kill switch; data-quality checks before any stock gets money.
Books	Shadow ledger (portfolio.py)	Our own copy of cash, margin, positions, average cost, STCG/LTCG per financial year, set-off and carry-forward. Re-synced with /portfolio.
Books	Independent audit (audit.py)	A second, separate implementation of the books and tax, used only to check the ledger.
Output	REST client (api.py)	Token bucket, anti-flood pacer, 429 penalty box, timeout classification, partial fills.
Output	Journal + dashboard	logs/*.jsonl and a Streamlit dashboard that reads only local logs.

Technologies used

- **Python 3** (Anaconda) for everything.
- **kafka-python** for the market stream; **requests** for the REST gateway; **python-dotenv** for the secret token in .env.
- **pandas + Streamlit** for the live dashboard; standard library only (math, statistics, threading, concurrent.futures, dataclasses) for the trading logic, so the core has no heavy dependencies.
- **unittest** for 84 unit, integration and end-to-end tests; **multiprocessing** for parallel research sweeps; **Git / GitHub** (private repo) for version control.

5. How each part works

5.1 The core: 99% equal weight, bought on day 1

- On the first tradable day the bot buys every stock with the same amount: 99% of wealth ÷ 10 stocks, about ₹99,000 each. All 10 orders go in the same day (limit 10 orders/day).
- 0.5% (about ₹5,000) stays as cash for fees. A whole 500-day run's fees were ₹1,630, so this buffer is enough and orders never fail for lack of cash.
- The core is **held through dips**. Selling on a drawdown was tested and cost 9 points (it sold near lows).
- Sizing uses the lower of cost and current price, so a winner is never trimmed just because it went up (only the concentration cap trims winners).
- Holding more than 365 days turns gains into long-term capital gains (10% with a ₹1 L yearly exemption) instead of short-term (15%).

5.2 Concentration cap (30%)

- If one stock grows above **30%** of total wealth, the bot sells only the extra part, back to **26%**. The core never re-buys a stock above 26%.
- If the holding is older than 365 days, it is trimmed at once. The order log records the gain, how much fits in the ₹1 L LTCG exemption, and the estimated tax.
- If it is younger, the bot waits for the 365-day mark (cheaper tax) unless the stock goes above the hard limit of **35%**.
- Conservative rule: a holding counts as long-term only if even its *latest* purchase is over 365 days old, because we could not verify how the platform dates a partial sale of shares bought on different days.
- Why it exists: one stock (PLNX) rose about 13× in the data. Without a cap, a single stock would dominate the portfolio. The cap costs about 5 points of mean return and lowers drawdown by about 1 point; 25% would cost about 9 points, 35% about 2. We chose 30% as a balance of risk and return.

5.3 Tax engine (Indian capital gains rules as used by the platform)

- Tracks gains per Indian financial year (April–March).
  - **STCG** (held ≤ 365 days): 15%. **LTCG** (held over 365 days): 10%, first ₹1,00,000 per year exempt (Section 112A). Holding exactly 365 days is short-term; 366 is long-term. Short covers are always short-term.
  - **Section 70**: within a year, short-term losses offset long-term gains.
  - **Section 74**: losses that cannot be used are carried forward to later years.
- **Cost basis = average cost**, exactly like the platform. Verified: the platform's realized P&L on the live run sums to ₹77,058.03; our average-cost replay gives ₹77,046.57 (FIFO would give ₹1,21,572).
- **Tax-loss harvesting** from 15 to 31 March: if the year still has taxable gains, the bot sells a losing position and buys it straight back. The position is unchanged, the loss is booked and cancels taxable gain. India has no wash-sale rule. It costs about ₹55 of fees and runs only if the saving is more than 4× the fees.
- **LTCG deferral**: a soft exit of a winner within 30 days of its 365-day mark is postponed.
- On restart, realized gains for each year are reloaded from GET /tax/report, so nothing is forgotten.

5.4 Risk guard and kill switch

Condition	Effect
Drawdown ≥ 8% from the 60-day peak	New positions at half size
Drawdown ≥ 15%	No new positions; shorts closed
One-day loss ≥ 3%	No new positions for 2 days
File STOP in the bot folder	Exits only, no new buys (checked every simulated day, no restart)
File FLATTEN in the bot folder	Close every position
3 rejected orders in a row	Order circuit breaker: pause 60 s and resync with the server

5.5 Universe screen (data quality before money goes in)

Before a stock gets new money it must have: at least 120 bars of history; median turnover of at least ₹50 L/day; at most 5% zero-volume days; no 40% one-day jumps (bad tick or corporate action); no data gaps over 10 days; ATR at most 10%; and a recent last bar. A stock listed mid-run is discovered from the feed or GET /stocks, warm-started with history and screened. A failing stock can still be sold.

5.6 Orders: zero-rejection design

- Every order is checked against the shadow ledger using the gateway's own rules (cash incl. ₹20 brokerage, ₹15.34 DP charge on long sells, quantity held, 100% short margin) before it is sent.
- Entries are checked against a running cash budget so several buys on the same day cannot overspend.
- Exits go before entries. A (day, stock, action) is sent at most once per day (duplicate guard).
- The ledger is re-synced with GET /portfolio every 3 days and after any anomaly.

5.7 Failure handling

Situation	What the bot does
Connect timeout (never reached gateway)	Retried once
Read timeout, connection reset, HTTP 5xx	The order may have executed, so it is looked up in GET /orders twice. Found: booked. Not found: marked UNKNOWN, not booked, **never resent**, rest of the day halted, resync from /portfolio.
Partial fill	Only the filled shares are booked; resync
429 rate limit / auth / not verified	Pause 30–60 s; token errors disable orders with a clear log line
Kafka disconnect	Automatic reconnect with back-off; the day's batch continues
Bot restart	Warm start from /history, positions from /portfolio, tax from /tax/report

5.8 Request budget (rate limits)

- Platform limits: 120 requests/minute and an anti-flood limit of 15 requests per 5 seconds.
- Our limits: token bucket at 100/minute and a pacer at 12 per 5 seconds, always below the platform's.
- Heartbeat ACKs and /stage polls are marked **essential**, so our own budget never skips them. Public endpoints (/stage, /history) don't use the token.
- Result: in the live run, 0 rate-limit hits and 0 errors.

5.9 Pre-flight heartbeat and the switch to simulation

- During pre-flight the platform sends numbered heartbeats; the bot ACKs every sequence number in order, drops repeats within 30 s and retries failed ACKs. GET /bot/status confirms the platform sees us as READY.
- Day 1 of the simulation can arrive while the stage still says pre-flight. The bot decides that day without trading, and when the stage switches it trades that day at once if it is still current.
- **Bug we found and fixed:** in the live SimTest, at the switch our own request budget skipped 5 history downloads, so those stocks could not trade for about 5 days. Now the bot keeps the history it already has, waits its turn instead of skipping, and /stage polls are essential.

5.10 News sentiment

- Rule-based lexicon scoring of each headline and description, with negation handling ("no fraud" is not negative), averaged per stock per day with a 2-day half-life.
- We measured it: news does **not** predict the next day's price ($t = 0.56$; $|t|$ below 2 is chance). It moves with the same day's price ($t = 2.48$), which is too late to trade. 56% of headlines score zero and most are unrelated world news. So news is used only in the tactical profiles and for explanations.

5.11 Shorts, margin and why Margin Locked is 0

- A short needs a stock we don't hold. The core holds all 10, so no short can open and margin stays 0. That is a deliberate decision.

- The **hedged** profile frees the weakest stock in a broad sell-off (≤ 30% of stocks above their 100-day EMA, confirmed downtrend), sells its core, SHORTs it on a bounce (7% of wealth, 100% margin locked) and COVERs on its stop, when the bounce fades, when the downtrend ends, or after 5 days (the platform squares off shorts every 10 days).
- In the live replay it made 10 shorts and 10 covers but finished 4.4 points behind, without lower drawdown. Losers in this data rebound, so shorting them costs money. The code is built, tested and one setting away.

5.12 Strategy profiles (one setting in .env: STRATEGY_PROFILE)

Profile	What it is	When to use
core (default)	99% equal-weight core + 30% cap + tax engine + risk guard	Best return in all tests
active	85% core + tactical overweights (score ≥ +0.30, volatility-sized, 2.5 ATR stops)	If judges want more trading activity
hedged	active + short hedge in sell-offs	Shows shorts and margin in use
tactical	Signal timing only, about 50% invested; 51 shorts in 2025-26	Lowest drawdown, much lower return

5.13 Explainability: every order says why

Every order in logs/orders.jsonl carries an **explain** record: the signal score and its parts (regime, trend, mean reversion, momentum, news), confidence, sizing (target value, order value, what limited it), the risk checks passed, the reason in plain words, latency, fill price and order id. For a cap trim it also shows the gain, the LTCG exemption left and the estimated tax. logs/decisions.jsonl shows every stock's score every day, even on days with no orders.

5.14 Data growth and replay

- Every run's ticks and news are recorded. When the bot exits, the new prices are merged into data/history (only if they agree with existing prices) and each run is saved separately in data/runs/.
- **python backtest.py --replay-live** replays the recorded 500 SimTest days with the recorded headlines and prints the leaderboard's columns next to the live numbers.
- **python backtest.py --episodes** scores the current settings on all 32 live-length games.

5.15 Security

- The access token lives only in .env, which is git-ignored and excluded from the submission ZIP.
- package.py refuses to build if any file contains a JWT-like string, a token, a Bearer credential, a password assignment or the live token value. verify.py checks this too.
- The dashboard reads only local logs; it never calls the gateway.

6. How we chose the strategy (the evidence)

Our method was the same for every idea: build it, test it on all 32 live-length episodes (not one lucky period), keep it only if it improved the average *and* the worst case after fees and tax.

Idea tested	Result over 32 episodes / replay	Decision
Tactical timing layer (trend + mean reversion + momentum + news)	−5 to −10 points of mean return	Off by default (profile active)
80% core + tactical (what ran live)	+93.0% mean, worst +30.1%, 47 orders	Replaced
99% equal-weight core + 30% cap	+101.5% mean, worst +42.6%, 23 orders	**Chosen**
No concentration cap (97% core)	+104.6% mean, but one stock can dominate	Rejected for risk
Cap at 25% / 35%	−9 points / −2 points vs no cap	30% chosen as balance
Inverse-volatility weights (p = 1 / 0.5)	+65.4% / +84.4% mean	Rejected
Short hedge in sell-offs (hedged)	−4.4 points in replay, drawdown not lower	Built, off by default
Selling core on drawdown	−9 points out of sample	Rejected
News as a trading signal	Next-day t = 0.56 (no predictive power)	Explanation only
97% core / 2% cash vs 99% / 0.5%	+99.9% vs +101.5%	99% chosen

Key message: in a 10-stock market that rose strongly over these years, every timing rule spends money on fees and taxes and misses part of the rise. The scientific result was that the best decision is to be fully invested, diversified, tax-efficient, and capped against concentration.

7. Testing and verification

- **84 automated tests** (python -m unittest discover -s tests): indicators, strategy, planner, tax rules, ledger, risk guard, kill switch, hedge, concentration cap, cash reserve, pre-flight switch, replay, and an end-to-end run with a fake Kafka stream and fake exchange.
- **verify.py: 22 checks**, including: tests pass; no token in .env.example; .env git-ignored and not shipped; secret scan; parameter sanity; no kill-switch file left behind; price history and universe screen; accounting and tax audit for each profile; metrics consistent with the equity curve; and the replay reproduces the live SimTest wealth within 1%.
- **Independent audit**: audit.py re-implements the books and tax separately and matches the ledger on 300 random multi-year cases, and every backtest's books to the paisa (cash, margin, positions, fees, tax, wealth).
- **Live evidence**: SimTest live run 60 orders 0 errors; earlier sandbox runs 65+ orders including SHORT/COVER, 0 rejections, ledger matched /portfolio at every reconcile; a 1-share short round trip matched the server to the paisa.

8. Live demo plan for the judges

#	Show	Command	What to say
1	Health check	python verify.py	22 checks: tests, secrets, audit, replay match
2	Live game replay	python backtest.py --replay-live	Same columns as the leaderboard; matches live
3	Robustness	python backtest.py --episodes	All 32 games: mean, median, worst
4	Dashboard	streamlit run dashboard.py	Live wealth, positions, decisions, orders
5	Why an order happened	open logs/orders.jsonl	Explain record: signal, sizing, checks, reason
6	Kill switch	create a file named STOP	Bot switches to exits-only within one day
7	Running bot	python run_bot.py	Stage, heartbeats, decisions every 10 s

Keep the official bot running in its own terminal; run demos in a second terminal. Never press Ctrl+C in the live bot's terminal.

9. Questions the judges may ask, with our answers

Answer short first, then offer detail. If a judge points out a weakness, agree and show the data.

A. Strategy

Q1. In one sentence, what does your bot do?

It invests 99% of the money equally in all tradable stocks on day 1, holds them, trims any stock that grows past 30% of the portfolio, and manages every sale for the lowest legal tax, fully automatically and with zero order errors.

Q2. Isn't this just buy-and-hold? Why is it a bot?

The strategy is simple on purpose, but the bot is not. Buy-and-hold was the *result* of testing, not our starting point. Everything around it is autonomous decision-making:

- detects the stage and answers heartbeats; trades only in the official simulation;
- warm-starts history, screens data quality, discovers new stocks;
- sizes 10 orders to 99% within fees and the rate limit, executed in about 0.4 s;
- decides every day whether to trim, harvest tax losses, or hold, and logs why;
- risk guard, kill switch, timeout handling without double orders, reconciliation.
- Most days the correct decision is "hold". Knowing when not to trade is the skill.

Q3. Why equal weight instead of picking the best stocks?

Picking winners needs a signal that predicts. We tested momentum, trend, mean reversion and news: none predicted well enough to beat costs. Equal weight is the most robust choice when no signal is reliable, and it gave the best worst-case result (+42.6% in the worst of 32 games). Volatility weighting was also tested and was worse (+65% to +84% mean).

Q4. Why 99% invested and not 100%?

The last 0.5% (about ₹5,000) pays brokerage and DP charges so that no order can ever fail for insufficient cash. A whole run's fees were ₹1,630, so the buffer is enough and costs almost nothing.

Q5. Why buy everything on day 1? Isn't that risky?

Over these years prices drifted up; every day in cash missed that rise. Our tests showed waiting or timing the entry lowered the average result. The risk is spread over 10 stocks, the cap limits any one stock, and the risk guard stops new buying in a drawdown.

Q6. Why did the 1st-place team beat you?

They concentrated in 3 stocks that happened to rise the most. That is a bigger bet: it wins when the bet is right and loses heavily when wrong. They also realized ₹7.34 L of profit mostly short-term and paid ₹63,435 tax. We were diversified across 10 stocks with ₹0 tax, ₹1,630 charges and 0 errors. One 500-day game is noisy; across 32 games our approach gives a strong average and a good worst case.

Q7. How do you know you are not overfitting?

Three reasons. (1) Very few parameters: equal weight, a 30% cap, a 365-day tax rule. (2) We judge every setting on 32 different 500-day games, not one period, and look at the worst game as well as the average. (3) The backtest reproduces the live run within 0.14%. Honest limit: all data is now used for choosing settings, so there is no untouched hold-out period left.

Q8. What does the concentration cap do and why 30%?

No stock may be more than 30% of wealth; above that, the bot sells the excess back to 26%, preferably only after the holding is long-term for lower tax. We measured each level: 25% cost about 9 points of mean return, 30% about 5 (with about 1 point less drawdown), 35% about 2. 30% balances risk and return.

Q9. When does the bot sell?

Only for a reason: (1) the concentration cap, (2) March tax-loss harvesting (sell and buy back at once), (3) the FLATTEN kill switch, and in the active/hedged/tactical profiles, (4) stops and signal exits. Selling without a reason creates fees and tax.

Q10. What if a new stock is listed in the middle of the game?

The bot discovers it from the stream or GET /stocks, downloads its history, and runs the universe screen. If it passes, it becomes eligible for the core; if not, it gets no money.

B. Risk

Q11. What was your maximum drawdown and isn't it high?

About 24–25% (24.3% in the SimTest replay, 24.9% average over 32 games), roughly the same as the market, because we hold the market. We accept that to keep the return. The risk guard halves new buys at 8% drawdown and stops them at 15%; the FLATTEN switch can exit everything. Selling the core in drawdowns was tested and cost 9 points because it sold near the lows.

Q12. Isn't unrealized profit risky?

Yes: unrealized profit can fall if prices fall. But it already counts in total wealth, and selling it to "lock it in" would trigger tax, fees and miss further rises. We control the risk instead with diversification and the 30% cap, which takes profit from any stock that becomes too large.

Q13. What if the market crashes?

The risk guard stops new buying; the core is held (tested to be better than panic selling); STOP or FLATTEN give manual control; and the hedged profile can short the weakest stock in a broad sell-off.

Q14. Why are there no shorts and why is Margin Locked zero?

A short needs a stock we do not own, and the core owns all 10. We built and tested a short hedge: it made 10 shorts and 10 covers in the replay but lost 4.4 points without lowering drawdown, because the losers in this data rebounded. So shorting is a tested option, not our default.

C. Tax and costs

Q15. Why was your tax ₹0 when you made ₹77,058 of realized profit?

Every rupee was legally exempt or offset:

- FY 2024-25: small trims plus the March tax-loss harvest gave a net loss of about ₹4k, carried forward.
- FY 2025-26: about ₹59k was long-term gain (held over 365 days), below the ₹1 L LTCG exemption; the small short-term gain was cancelled by harvested and carried-forward losses.
- FY 2026-27: an ₹18k long-term gain, again below the exemption.
- It was by design: hold over 365 days, harvest losses every March, rarely sell winners.

Q16. What is tax-loss harvesting? Is it legal?

Selling a losing position and buying it straight back to book the loss for tax, keeping the same holding. India has no wash-sale rule, so it is legal. The bot does it only in late March, only when the year has taxable gains, and only when the saving is more than 4× the fees (about ₹55).

Q17. How does your bot calculate tax?

A shadow ledger tracks every lot with average cost (matching the platform: ₹77,058.03 on the platform vs ₹77,046.57 in our replay), STCG 15%, LTCG 10% with ₹1 L exemption, set-off (Sec 70) and carry-forward (Sec 74) per financial year. An independent implementation checks it on 300 random cases.

Q18. Why so few orders?

Each order costs ₹20 brokerage plus ₹15.34 DP on sells, and each sale can create tax. Fewer, purposeful orders mean lower costs and zero chance of errors. We placed 60 orders live (11 in the improved replay); teams with 400–723 orders paid ₹10k–20k in charges.

D. Engineering

Q19. How fast is your bot?

About 0.35 s per decision on average (0.51 s on the live leaderboard), against a 10-second day. It decides as soon as all ticks are in and news has been quiet for 150 ms.

Q20. How do you guarantee zero order errors?

Every order is pre-validated against our own copy of the account using the gateway's rules, entries respect a running cash budget, a duplicate guard allows one (day, stock, action), we stay under the rate limits, and the ledger is re-synced with the server. Live: 60 orders, 0 errors.

Q21. What happens if an order times out?

If the request never reached the gateway, it retries once. If it might have executed, the bot checks the order list twice. If it still cannot confirm, it marks the order UNKNOWN, never resends it, stops trading for the day and resyncs. This prevents double orders.

Q22. What if Kafka disconnects or the bot restarts?

Kafka reconnects automatically with back-off. On restart, history comes from /history, positions from /portfolio and tax from /tax/report, so the bot continues correctly. Live downtime was 0 s.

Q23. Why a single-threaded engine?

One thread owns the portfolio state, so there are no race conditions or double spending. Network work runs in parallel around it: Kafka consumers, the stage watcher, the heartbeat worker and a pool of up to 10 order workers.

Q24. How does the heartbeat / readiness work?

In pre-flight the platform sends numbered heartbeats. A dedicated worker ACKs every sequence in order, drops duplicates, retries failures and checks GET /bot/status. Its requests bypass our local budget so they are never delayed.

Q25. Did you find any bugs in live testing?

Yes. At the switch from pre-flight to simulation, our own rate limiter skipped 5 history downloads, so 5 stocks could not trade for about 5 days. We found it by replaying the live run, fixed it (keep existing history, wait instead of skip, stage polls essential), and added a test.

Q26. How did you test the bot?

84 automated tests, a 22-check verify script, an independent accounting and tax audit, a replay that matches the live run within 0.14%, and live sandbox runs with real orders including SHORT and COVER.

Q27. How do you keep the access token safe?

It is only in .env, which git ignores and the packager excludes; the packager and verify script scan every shipped file for tokens and refuse to build if one is found.

Q28. What technologies did you use?

Python 3, kafka-python, requests, python-dotenv, pandas and Streamlit for the dashboard, unittest, multiprocessing for research, and Git/GitHub. The trading logic uses only the standard library.

E. Machine learning and AI

Q29. Do you use machine learning?

No, and that is a deliberate, evidence-based choice. The bot is rule-based and fully explainable. Its parameters were chosen by systematic data-driven testing over 32 episodes, and its news reading is rule-based NLP (a sentiment lexicon with negation handling).

Q30. Why not ML? Isn't that the modern approach?

Because the data and the problem do not support it:

- **Too little data:** 10 stocks × about 4 years is roughly 10,000 data points and one market path. A model would learn noise and overfit.
- **No signal to learn:** our tests show news does not predict next-day prices (t = 0.56) and every price-timing rule lost money after costs. ML cannot find a signal that is not there.
- **Explainability:** the jury must understand every trade. Each of our orders states its reason; an ML model would be a black box.
- **Reliability:** a simple, tested rule cannot misbehave on unseen data the way a model can.
- We used the scientific method that ML is meant to serve: hypothesis, test on many episodes, keep only what works.

Q31. Could you add ML without hurting performance?

Yes, as a **shadow advisor**: a model makes predictions alongside the bot and logs them, but trades only if it beats the current core on all 32 episodes. That adds innovation with zero risk to results.

Q32. Is your news sentiment AI?

It is rule-based natural language processing: a weighted word list with negation handling, averaged per stock with a 2-day half-life. We measured it honestly and found it weak, so it is used for explanations and tactical profiles, not for the default core.

F. Results, honesty and the hackathon

Q33. How did you do in the live test?

2nd on the leaderboard: ₹14,40,868 (+44.1%), 0 errors, 0 downtime, 0.51 s decisions, ₹1,630 charges, ₹0 tax, 88.5% cash utilization, 10 positions, 60 orders.

Q34. Your improved bot replays slightly below the live result. Why?

The live run benefited from a lucky data glitch (5 stocks entered about 5 days late at better prices). Our simulator reproduces that glitch within 0.14%. Without the glitch, the settings that ran live give ₹13.34 L and the current bot ₹14.37 L, so the current bot is about 10 points better on equal terms.

Q35. What are your bot's weaknesses?

Section 10 lists them honestly: drawdown similar to the market; plain 100% buy-and-hold beat us by about 8 points on average because of the cap and the cash buffer; all data is used for tuning; and 10 stocks over 4 years is a small sample.

Q36. Why is cash utilization below 100%?

We keep 0.5% for fees, day 1 of the simulation is observed (orders start on the first tradable day), and cap trims return cash. Average utilization over 32 games is 98.7%; it was 88.5% live with the old 80% core.

Q37. What would you do with more time?

A live strategy tournament (all profiles run in paper mode side by side, best one shown on the dashboard), a Monte Carlo risk report, an ML shadow advisor, an anomaly detector for bad data, and an automatic end-of-run report.

Q38. Why should we choose your team?

Because our bot is the complete package the challenge asks for: strong return, zero errors, near-zero costs and tax, full autonomy, explainable decisions, and every claim backed by tests and data, including honest reports of what did not work.

10. Honest limitations and what we would do next

Limitations (say these before a judge does)

- **Drawdown is market-like** (about 25%) because we stay invested. We chose return over a smaller drawdown and showed the trade-off.
- **Plain 100% buy-and-hold beat us by about 8 points on average** over 32 games. The gap is the 0.5% cash buffer and the 30% cap selling part of the biggest winner (PLNX, about 13× in the data). We accept that cost as risk control.
- **No untouched hold-out period**: all data is now used to choose settings.
- **Small sample**: 10 stocks over about 4 years, essentially one market path.
- **Partial-sale holding period**: how the platform dates a partial sale of shares bought on different days could not be verified, so we use the conservative rule.
- **No shorts in the default**: they are built and tested but cost return in this data.

Next steps / innovation ideas

Idea	What it adds
Live strategy tournament	All profiles run as paper portfolios on the live stream; the dashboard ranks them in real time
Monte Carlo risk report	Thousands of resampled market paths to show the range of outcomes, not one number
ML shadow advisor	A model that predicts and logs but trades only if it beats the core on all episodes
Data anomaly detector	Flags bad ticks or impossible prices before they reach the strategy
Tax-free gain harvesting	Uses the unused ₹1 L LTCG exemption each year by selling and re-buying long-term winners
Automatic end-of-run report	A PDF/HTML report with the leaderboard columns, tax by year and every decision
"Why not" explanations	Logs why the bot did *not* trade a stock each day, not only why it did

11. Glossary

Term	Meaning
Total wealth	Cash + value of holdings at current prices + locked margin
Realized profit	Profit (or loss) from shares actually sold or shorts covered
Unrealized profit	Gain on shares still held, at today's price; not taxed until sold
Charges	Fees: ₹20 brokerage per order, plus ₹15.34 DP charge on each long sell
Cash utilization	Share of wealth that is invested rather than sitting in cash
Margin locked	Cash blocked as collateral for open shorts (100% of the short value)
POS	Number of open positions (different stocks held) at the end
Orders / errors	Orders executed / orders rejected or failed
Downtime	Time the bot was disconnected from the stream
Decision time	Time from the day's ticks arriving to the decision
SHORT / COVER	Sell shares you don't own hoping to buy back cheaper / buy them back to close
STCG / LTCG	Short-term (≤ 365 days, 15%) / long-term (over 365 days, 10%, ₹1 L yearly exemption) capital gains
Set-off / carry-forward	Using losses to cancel gains in the same year (Sec 70) / in later years (Sec 74)
Tax-loss harvesting	Sell a losing holding and re-buy it to book the loss for tax
Drawdown	Fall from the highest wealth reached to a later low
Core	The long-term equal-weight holding of every stock
Concentration cap	Upper limit on one stock's share of wealth (30%, trimmed to 26%, hard limit 35%)
EMA	Exponential moving average, a smoothed price trend
ATR	Average true range, a stock's typical daily price movement (volatility)
RSI / Bollinger z	Overbought/oversold measures used by the mean-reversion part of the score

Term	Meaning
Breadth	Share of stocks above their 100-day EMA (how healthy the whole market is)
Episode	One 500-day test game in the data, with 400 days of history before it
Kafka	The streaming system that delivers ticks and news to the bot
Heartbeat	Numbered messages in pre-flight; the bot must acknowledge each to be READY
Shadow ledger	The bot's own copy of the account, used to pre-check orders
PLNX	One of the 10 masked stock tickers; the biggest winner in the data (about 13×)

12. One-page numbers cheat sheet

Number	Value
Live SimTest rank	2nd
Live wealth / return	₹14,40,868.47 / +44.1%
Live realized / unrealized	₹77,058 / ₹3,65,453
Live charges / tax	₹1,629.52 / ₹0
Live orders / errors / downtime	60 / 0 / 0 s
Live decision time	0.51 s average (about 0.35 s in current runs)
Live cash utilization / positions	88.5% / 10
Replay accuracy vs live	within 0.14% (₹14,38,801 vs ₹14,40,868)
Current bot on SimTest replay	₹14,36,954 (+43.7% after tax), 11 orders, ₹235 charges
Old live settings on replay	₹13,33,500 (+33.4%), 42 orders
32 episodes: mean / median / worst	+101.5% / +72.6% / +42.6%
32 episodes: drawdown / utilization	24.9% / 98.7%
32 episodes: orders / tax	about 23 / ₹4,658 average
Core / cash buffer	99% equal weight / 0.5% (about ₹5,000)
Concentration cap	30% trigger, trim to 26%, hard limit 35%
Orders per day / rate limits	10 / 100 per min and 12 per 5 s (platform: 120 per min, 15 per 5 s)
Risk guard	8% drawdown half size, 15% no new buys, 3% daily loss 2-day pause
Tax rules	STCG 15%, LTCG 10% over 365 days, ₹1 L LTCG exemption, Sec 70/74/112A
Fees	₹20 per order + ₹15.34 DP per long sell
News predictive power	next-day t = 0.56 (not predictive); same-day t = 2.48
Tests / checks	84 automated tests / 22 verification checks
Platform cost-basis check	₹77,058.03 platform vs ₹77,046.57 ours (average cost)

If you remember only three things: (1) 2nd place with 0 errors, ₹0 tax and the lowest costs. (2) The strategy was chosen by testing every idea on 32 games, not by guessing. (3) Every order is pre-checked and explained, and the simulator matches the live result within 0.14%.